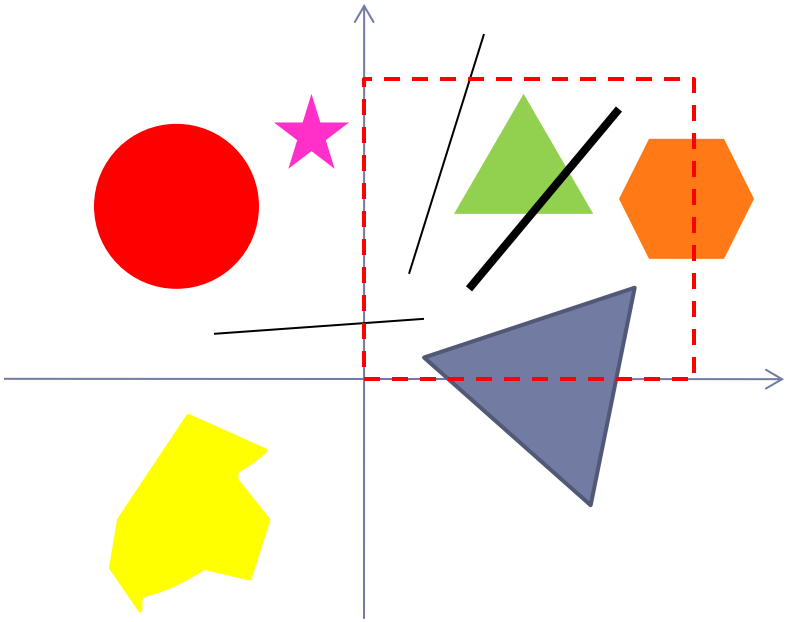
Clipping

GBC204 – Computação Gráfica
Prof. Dr. rer. nat. Daniel D. Abdala

Problematização

- Considere um mundo complexo composto por diversas primitivas gráficas;
- Considere ainda que o mundo é muito grande para ser todo apresentado de uma vez ou que tem-se interesse em visualizar apenas uma parcela dele;

- Processar todo o conteúdo é custoso e, em última instância, desnecessário;
- Solução: Processar apenas a parcela do mundo que efetivamente será visualizada.



Clipping

- O processo de seleção das primitivas (ou parte delas) é chamado de clipping (recorte);
  - Decidir quais primitivas geométricas encontram-se localizadas totalmente ou parcialmente;
  - Detectar Intersecção de primitivas;
  - Calcular sombras (no caso 3D);
- É um processo de Otimização fundamental em CG;
  - Pre-processamento de visibilidade;
  - Remove potencialmente parte consideravel do mundo antes de se realizar os cálculos de visualização;

Clipping Window

- Região válida para rasterização;
  - Geralmente retangular;
  - Normalmente corresponde a window;
- Tipos de Clipping:
  - **Recorte de pontos (point clipping);**
  - **Recorte de linhas (line clipping);**
  - Recorte de áreas e polígonos (area and polygon clipping);
  - Recorte de curvas (curve clipping);
  - Recorte de texto (text clipping).

Recorte de Pontos

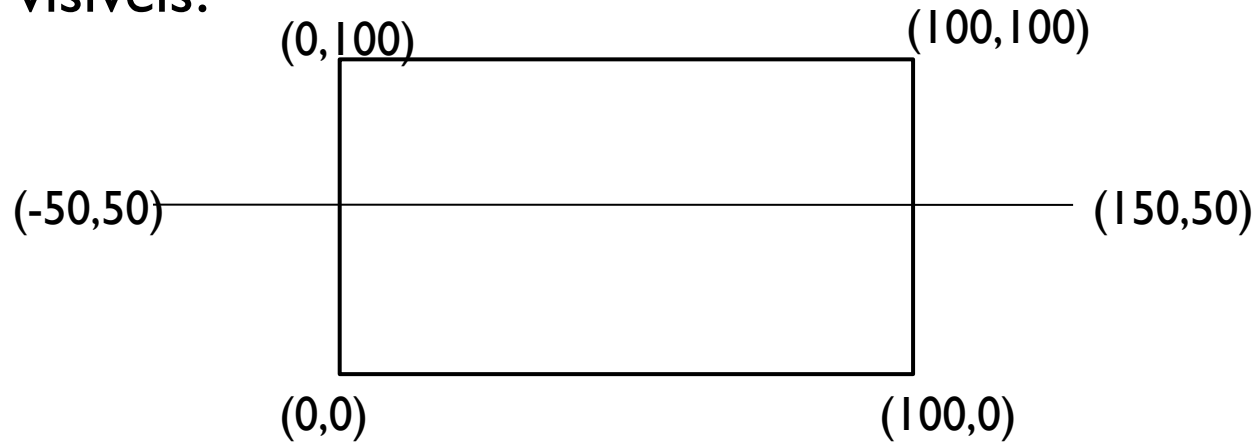
- Implementação trivial;
- Basta resolver as duas inequações abaixo. Caso o ponto P(x,y) atenda ambas, o ponto encontra-se dentro da área de clipping e deve ser rasterizado.

$$x_{W\min} \leq x \leq x_{W\max}$$

$$y_{W\min} \leq y \leq y_{W\max}$$

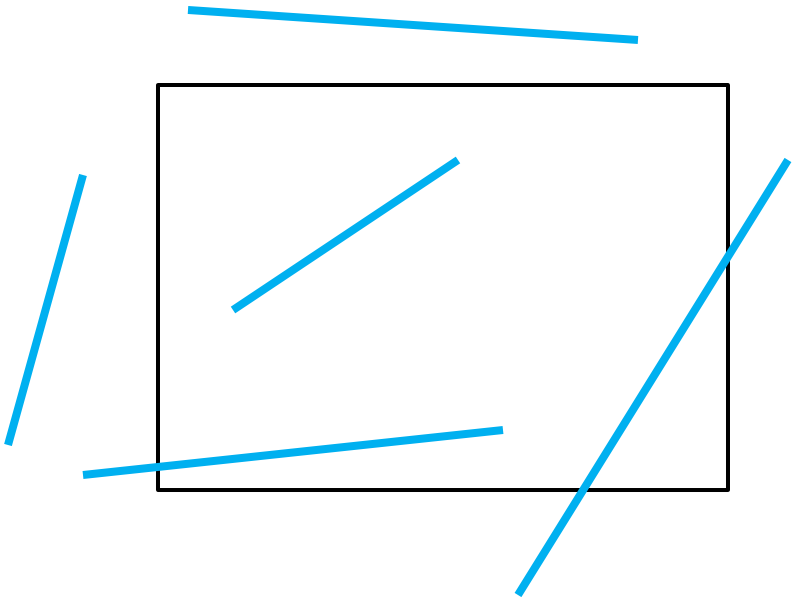
Recorte de Linhas

- Problema Fundamental: Uma linha pode ter ambos pontos inicial e final localizados fora do viewport e ainda sim possuir partes visíveis!

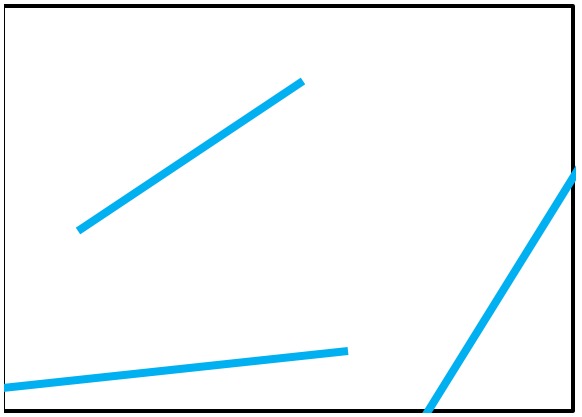


- Solução: Método Sistemático para:
  - Rejeitar Linhas totalmente fora;
  - Aceitar linhas totalmente dentro;
  - Demais: Cortar iterativamente até que se encaixem em um dos casos anteriores!

Efeito do Clipping de Linhas



Antes do Clipping



Depois do Clipping

Algoritmo: Cohen_Sutherland

- Método rápido e computacionalmente pouco intensivo;
- Decide rápido entre aceitar ou rejeitar uma linha;
- Usa um esquema de posicionamento por setores

1001	1000	1010
0001	0000 window	0010
0101	0100	0110

- Bit 1: acima (y > ymax)
- Bit 2: abaixo (y > ymin)
- Bit 3: direita (y > ymax)
- Bit 4: esquerda (x > ymin)
- Se ambos cod 1 e cod 2 = 0000 : totalmente dentro
- Se (cod1 & cod2) != 0 totalmente fora
- Senão, escolher ponto fora, cortar na borda mais distante e repetir

compute_code.py

```
# Cohen-Sutherland para linha
INSIDE = 0  # 0000
LEFT   = 1  # 0001
RIGHT  = 2  # 0010
BOTTOM = 4  # 0100
TOP    = 8  # 1000

def computeCode(x, y):
    code = INSIDE
    if x < xmin: code |= LEFT
    elif x > xmax: code |= RIGHT
    if y < ymin: code |= BOTTOM
    elif y > ymax: code |= TOP
    return code
```

Cohen-Sutherland (ideia geral)

```
def cohen_sutherland(x1, y1, x2, y2, xmin, ymin, xmax, ymax):
    # Passo 1: calcular códigos iniciais
    code1 = compute_code(x1, y1, xmin, ymin, xmax, ymax)
    code2 = compute_code(x2, y2, xmin, ymin, xmax, ymax)
    aceito = False
    terminado = False

    while not terminado:
        if code1 == 0 and code2 == 0:
            # Caso 1: ambos dentro → aceitar
            aceito = True
            terminado = True

        elif (code1 & code2) != 0:
            # Caso 2: ambos fora do mesmo lado → rejeitar
            terminado = True

        else:
            # Caso 3: um dentro, um fora, ou ambos fora de lados opostos
            # Escolher um ponto fora (code1 != 0)
            if code1 != 0:
                code_out = code1
            else:
                code_out = code2
```

Cohen-Sutherland (ideia geral)

```
            # Encontrar interseção com a borda apropriada
            x, y = intersect(x1, y1, x2, y2, code_out, xmin, ymin, xmax, ymax)

            # Substituir o ponto fora pelo ponto de interseção
            if code_out == code1:
                x1, y1 = x, y
                code1 = compute_code(x1, y1, xmin, ymin, xmax, ymax)
            else:
                x2, y2 = x, y
                code2 = compute_code(x2, y2, xmin, ymin, xmax, ymax)

    if aceito:
        desenhar_linha(x1, y1, x2, y2)
```

- Algoritmo completo?
- Considere os casos em que code1=0001 e code2 = 0001, ou o caso em que code1 = 1001 e code2 = 1010

```python
def compute_intersection(x1, y1, x2, y2, code, xmin, ymin, xmax, ymax):
    """
    Calcula o ponto de interseção entre a reta (x1,y1)-(x2,y2)
    e a borda da janela indicada pelo código.

    Retorna: (x_int, y_int) ponto de interseção
    """
    # Prevenir divisão por zero (linhas degeneradas)
    if x2 == x1 and y2 == y1:
        return x1, y1  # ponto único

    # TOP (acima) - código bit 8
    if code & 8:
        # Calcular t para y = ymax
        if y2 != y1:
            t = (ymax - y1) / (y2 - y1)
        else:
            t = 0  # linha horizontal no topo

        x = x1 + t * (x2 - x1)
        y = ymax
        return x, y

    # BOTTOM (abaixo) - código bit 4
    if code & 4:
        # Calcular t para y = ymin
        if y2 != y1:
            t = (ymin - y1) / (y2 - y1)
        else:
            t = 0

        x = x1 + t * (x2 - x1)
        y = ymin
        return x, y

    # RIGHT (direita) - código bit 2
    if code & 2:
        # Calcular t para x = xmax
        if x2 != x1:
            t = (xmax - x1) / (x2 - x1)
        else:
            t = 0  # linha vertical na direita
```

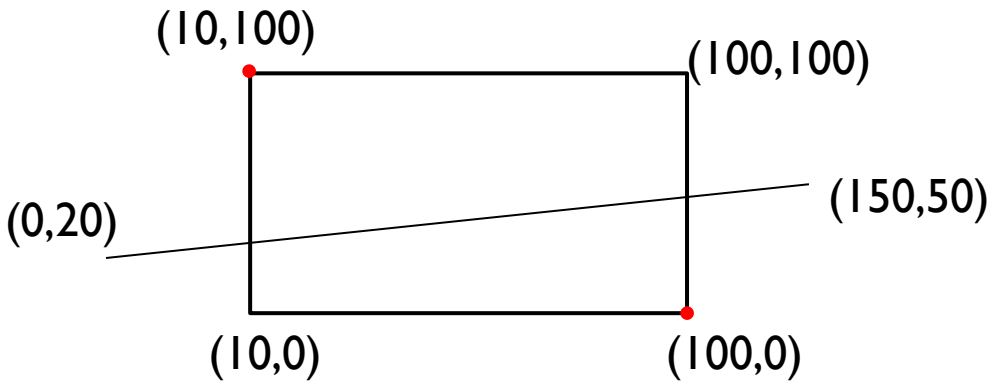
```
        x = xmax
        y = y1 + t * (y2 - y1)
        return x, y

    # LEFT (esquerda) - código bit 1
    if code & 1:
        # Calcular t para x = xmin
        if x2 != x1:
            t = (xmin - x1) / (x2 - x1)
        else:
            t = 0

        x = xmin
        y = y1 + t * (y2 - y1)
        return x, y

    # Se código = 0, ponto está dentro (não precisa cortar)
    return x1, y1
```

Exemplo



1001	1000	1010
0001	0000 window	0010
0101	0100	0110

Pontos a Considerar

- Clipping de
  - Polígonos ?
  - Curvas?